

Disciplina: Aprendizado Profundo

Aula 3: Feedforward Neural Networks e Backpropagation

Eliezer de Souza da Silva
sereliezer.github.io
eliezer.silva@ufc.br

Mestrado e Doutorado em Ciência da Computação, Universidade Federal do
Ceará (MDCC / UFC)

15 de Setembro de 2025

Roteiro da Aula de Hoje

- 1 Revisão e Motivação
- 2 Modelos Não-Lineares: Fundamentos
- 3 O Multilayer Perceptron (MLP)
 - Funções de Ativação
- 4 Treinando MLPs: Backpropagation
 - Grafo Computacional
 - Algoritmo de Backpropagation
- 5 Próximos Passos

Recapitulando a Aula 2

Onde Paramos

- Modelos lineares (Perceptron, GLMs) definem uma fronteira de decisão linear e falham em problemas não-linearmente separáveis como o XOR.
- A Descida de Gradiente é nossa ferramenta de otimização principal para modelos com custos diferenciáveis.
- Autoencoders lineares nos mostraram que é possível aprender features (como PCA) mesmo com modelos simples.

A Pergunta Central de Hoje

Como superamos a barreira da linearidade para aprender features e resolver problemas complexos?

A Solução: Múltiplas Camadas para Aprender Features I

A Ideia: Transformar o Espaço de Features

Se os dados não são linearmente separáveis no espaço original, talvez possamos mapeá-los para um novo espaço onde eles se tornem separáveis.

- Uma **camada oculta** de neurônios atua como um **extrator de features**, aprendendo a função de mapeamento $h = \phi(x)$.
- A camada de saída, então, opera sobre esta nova representação h , que foi projetada para ser mais “fácil” de trabalhar.

A Solução: Múltiplas Camadas para Aprender Features II

A Transformação Mágica

A camada oculta com 2 neurônios (e ativações ReLU, por exemplo) pode aprender a mapear os quatro pontos de entrada para um novo espaço 2D. Neste novo espaço, os pontos das classes 0 e 1 são *linearmente separáveis*. A camada de saída só precisa aprender essa linha simples.

Visualizando o Aprendizado de Features Interativamente I

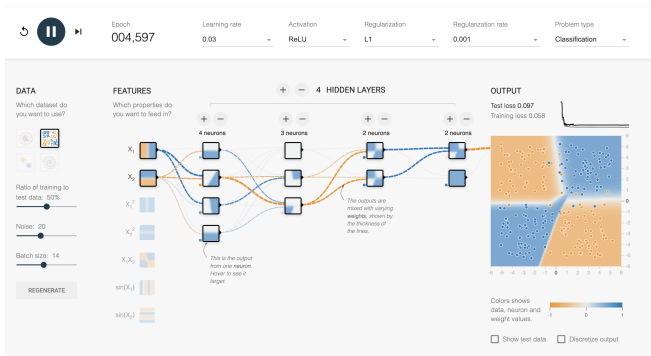


Figura: Uma rede com camadas ocultas aprendendo a fronteira de decisão complexa do problema XOR.

Visualizando o Aprendizado de Features Interativamente II

TensorFlow Playground

O TensorFlow Playground é uma ferramenta web interativa que nos permite construir, treinar e visualizar redes neurais em tempo real. É uma excelente forma de construir a intuição sobre como as camadas ocultas funcionam.

[Abrir Playground \(XOR\)](#)

Uma Rota Alternativa: O Truque do Kernel I

Aprendizado de Representação Explícito vs. Implícito

- **MLPs:** Aprendem um mapeamento **explícito** para o espaço de features ($h = \phi(x)$) e a função de decisão final simultaneamente.
- **Métodos de Kernel (e.g., SVM):** Usam o “truque do kernel” para operar em um espaço de features de alta (ou infinita) dimensão de forma **implícita**, sem nunca computar $\phi(x)$.

Uma Rota Alternativa: O Truque do Kernel II

A Função de Kernel

Uma função de kernel $k(x, z)$ calcula o produto escalar no espaço de features, $k(x, z) = \langle \phi(x), \phi(z) \rangle$, sem precisar do mapeamento ϕ . Isso permite o uso de espaços de features extremamente poderosos com custo computacional gerenciável.

Teorema do Representador I

A Forma da Solução em Métodos de Kernel

A solução $h^* \in \mathcal{H}$ que minimiza um funcional de risco regularizado sempre pode ser escrita como uma **combinação linear do kernel** avaliado nos pontos de treinamento: [Bernhard Schölkopf e Alexander J Smola \(2002\)](#). *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press

$$h^*(x) = \sum_{n=1}^N \alpha_n k(x, x_n)$$

Teorema do Representador II

Intuição da Prova

- 1 Qualquer função h no espaço de funções do kernel (RKHS) pode ser decomposta em uma parte que está no “span” ou espaço gerador pelos dados, $h_{\parallel}$, e uma parte ortogonal, $h_{\perp}$.
- 2 A parte ortogonal $h_{\perp}$ **não afeta** o erro nos dados de treino, mas **sempre aumenta** o termo de regularização ($\| \cdot \|_{\mathcal{H}}^2$).
- 3 Portanto, para minimizar o risco total (erro + regularização), a componente ortogonal deve ser zero ($h_{\perp} = 0$). A solução ótima deve ser uma combinação linear das funções de kernel centradas nos dados.

Fundamento: A Perspectiva da Teoria do Aprendizado Estatístico

Relembrando o Dilema Central

- Nosso objetivo é encontrar uma hipótese $h \in \mathcal{H}$ que minimize o **Risco Teórico** (erro sobre a distribuição desconhecida dos dados):

$$R(h) = \mathbb{E}_{(x,y) \sim p_{data}} [L(h(x), y)]$$

- No entanto, só temos acesso ao **Risco Empírico** (erro nos dados de treino $(x, y) \in \mathcal{D}_{\text{treino}}$):

$$\hat{R}(h) = \frac{1}{N} \sum_{n=1}^N L(h(x_n), y_n)$$

Do Risco Teórico ao Risco Regularizado I

1. O Objetivo Ideal: Minimizar o Risco Real

O objetivo final do aprendizado é encontrar uma hipótese h que generalize bem para dados não vistos, ou seja, que minimize o **Risco Teórico**:

$$R(h) = \mathbb{E}_{(x,y) \sim p_{data}} [L(h(x), y)]$$

Problema: Não conhecemos a verdadeira distribuição dos dados p_{data} , então não podemos calcular esta expectativa.

Do Risco Teórico ao Risco Regularizado II

2. A Aproximação Prática: Minimizar o Risco Empírico

A melhor aproximação que temos para o Risco Real é o **Risco Empírico**, a média do custo sobre nosso conjunto de treino:

$$\hat{R}(h) = \frac{1}{N} \sum_{n=1}^N L(h(\mathbf{x}_n), y_n)$$

Problema: Minimizar apenas $\hat{R}(h)$ pode levar ao *overfitting*, pois o modelo pode simplesmente memorizar os dados de treino, resultando em um grande *gap de generalização* ($R(h) \gg \hat{R}(h)$).

Do Risco Teórico ao Risco Regularizado III

3. A Solução de Engenharia: Minimizar o Risco Empírico Regularizado

Para combater o overfitting, adicionamos um termo de **regularização** $\Omega(\theta)$ que penaliza a complexidade do modelo:

$$J(\theta) = \hat{R}(h_\theta) + \lambda\Omega(\theta)$$

Minimizar $J(\theta)$ busca um balanço: encontrar um modelo que se ajuste bem aos dados de treino (baixo $\hat{R}$) sem se tornar excessivamente complexo (baixo $\Omega(\theta)$), o que promove uma melhor generalização.

Decompondo o Erro de Generalização I

As Três Fontes de Erro

O “excesso de risco”, a diferença entre o erro do nosso modelo treinado ($\hat{h}$) e o melhor erro possível (h^*), pode ser decomposto em três fontes principais, como aponta Bach (2025):

1. Erro de Aproximação

Nosso espaço de hipóteses $\mathcal{H}$ pode não ser rico o suficiente para conter a função “verdadeira”.

2. Erro de Estimação

A hipótese que minimiza o erro no treino ($\hat{h}$) não é a mesma que minimizaria o erro na população.

3. Erro de Otimização

O tipo de mínimo encontrado pelo otimizador e função de custo não-convexa.

Decompondo o Erro de Generalização II

O Teorema da Generalização

A teoria do aprendizado estatístico nos fornece um limite superior para o Risco Real (erro real) com base no Risco Empírico (erro no treino). Com alta probabilidade $1 - \delta$, para qualquer hipótese $h \in \mathcal{H}$ e n dados de treino:

$$\underbrace{R(h)}_{\text{Risco Real}} \leq \underbrace{\hat{R}(h)}_{\text{Risco Empírico}} + \underbrace{f(\mathfrak{R}_n(\mathcal{H}))}_{\text{Complexidade do Modelo}} + \underbrace{\sqrt{\frac{\log(1/\delta)}{n}}}_{\text{Termo de Confiança}}$$

Este é um dos resultados mais importantes da área (Vapnik 1998; Shalev-Shwartz e Ben-David 2014).

Decompondo o Erro de Generalização III

Complexidade de Rademacher ($\mathfrak{R}_n(\mathcal{H})$)

- É a ferramenta matemática que mede a “riqueza” ou “flexibilidade” do nosso espaço de hipóteses $\mathcal{H}$.
- Para redes neurais, ela depende não apenas do número de parâmetros, mas crucialmente das **normas dos pesos**. Modelos com pesos de norma menor são “mais simples”.
- Ela geralmente escala com $O(1/\sqrt{n})$, explicando a taxa de convergência mencionada.

Da Teoria à Prática: Motivando a Regularização I

O Dilema do Aprendizado

O teorema anterior revela um trade-off fundamental:

- Se nosso espaço de hipóteses $\mathcal{H}$ é muito **simples**, o *erro de aproximação* será alto (underfitting).
- Se nosso espaço $\mathcal{H}$ é muito **complexo**, o termo de *complexidade do modelo* ($\mathfrak{R}_n(\mathcal{H})$) será alto, levando a um grande *erro de estimação* (overfitting).

Da Teoria à Prática: Motivando a Regularização II

Regularização como um Proxy para a Complexidade

Não podemos minimizar diretamente a Complexidade de Rademacher. Em vez disso, adicionamos um termo de **regularização** $\Omega(\theta)$ à nossa função de custo, que atua como um *proxy* (uma aproximação) para a complexidade do modelo.

$$\hat{\theta} = \arg \min_{\theta} \left[\underbrace{\hat{R}(h_{\theta})}_{\text{Ajuste aos Dados}} + \underbrace{\lambda \Omega(\theta)}_{\text{Controle da Complexidade}} \right]$$

Controlar a norma dos pesos (e.g., com regularização L2, $\Omega(\theta) = \|\theta\|_2^2$) é uma forma eficaz de controlar a Complexidade de Rademacher, reduzir o erro de estimação e melhorar a generalização.

Minimização do Risco Empírico (ERM) Regularizado I

O Princípio do ERM

O treinamento de um modelo consiste em encontrar os parâmetros θ que minimizam o Risco Empírico, geralmente com um termo de regularização para controlar a complexidade e melhorar a generalização

$$\hat{\theta} = \arg \min_{\theta} \left[\underbrace{\frac{1}{N} \sum_{n=1}^N L(h(\mathbf{x}_n; \theta), y_n)}_{J(\theta)} + \lambda \Omega(\theta) \right]$$

Minimização do Risco Empírico (ERM) Regularizado II

Componentes

Definimos os componentes do risco empírico regularizado $J(\theta)$:

- $L(\cdot, \cdot)$ é a **função de custo (loss)**, que mede o erro para um único exemplo.
- $\Omega(\theta)$ é o **regularizador** (e.g., norma L_2 dos pesos, $\|W\|_F^2$).
- λ é o hiperparâmetro que controla a força da regularização.

Otimização via Descida de Gradiente I

A Regra de Atualização sobre o Risco Empírico

Para encontrar os parâmetros ótimos $\hat{\theta}$, usamos a Descida de Gradiente (geralmente estocástica com mini-batches) sobre a função de Risco Empírico Regularizado $J(\theta)$:

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \eta \nabla_{\theta} J(\theta^{(t)})$$

Otimização via Descida de Gradiente II

O Desafio Central: Calcular o Gradiente

O gradiente do risco é a média dos gradientes do custo para cada dado no mini-batch, mais o gradiente do regularizador:

$$\nabla_{\theta} J(\theta) = \left(\frac{1}{B} \sum_{n \in \mathcal{B}} \nabla_{\theta} L(h(\mathbf{x}_n; \theta), y_n) \right) + \lambda \nabla_{\theta} \Omega(\theta)$$

O nosso principal desafio técnico agora é: **como calcular eficientemente o gradiente $\nabla_{\theta} L$ para uma FNN/MLP?**

Resposta: Com o algoritmo de **Backpropagation**.

Definição: Feedforward Neural Network (MLP) I

O que é um MLP?

Um Multilayer Perceptron (MLP), ou Feedforward Neural Network (FNN), é uma função parametrizada que é formada pela **composição de múltiplas camadas** de transformações afins seguidas por funções de ativação não-lineares.

Definição: Feedforward Neural Network (MLP) II

Notação por Camada

Para uma rede com L camadas:

- A entrada para a camada l é a ativação da camada anterior, $a^{(l-1)}$.
- A entrada linear (pré-ativação) da camada l é:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

- A saída (ativação) da camada l é:

$$a^{(l)} = \sigma_l(z^{(l)})$$

Por definição, a ativação da “camada 0” é a entrada da rede, $a^{(0)} = x$, e a saída final é a ativação da última camada, $\hat{y} = a^{(L)}$.

Definição: Feedforward Neural Network (MLP) III

Uma Função Parametrizada

A rede inteira representa uma única função complexa h com parâmetros $\theta = \{W^{(1)}, b^{(1)}, \dots, W^{(L)}, b^{(L)}\}$:

$$\hat{y} = h(x; \theta) :$$

$$x \rightarrow z^{(1)} = W^{(1)}a^{(0)} + b^{(1)} \rightarrow a^{(1)} = \sigma_1(z^{(1)}) \rightarrow \dots \rightarrow \hat{y} = a^{(L)}$$

Anatomia de uma Rede Profunda

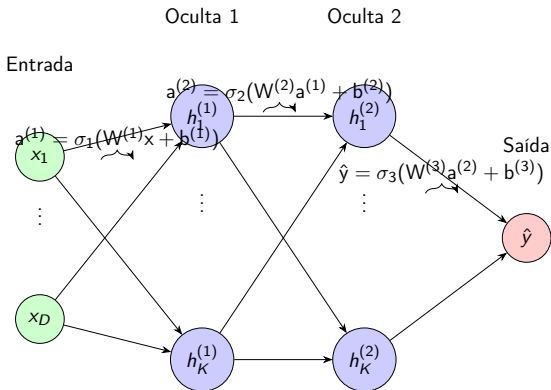


Figura: Visualização de um MLP com duas camadas ocultas. Cada camada transforma a representação da camada anterior.

O Teorema da Aproximação Universal (UAT) I

Quão Poderoso é um MLP com Uma Camada Oculta?

*Um FNN com uma **única camada oculta**, contendo um número finito de neurônios e uma função de ativação não-linear "esmagadora" (como a sigmóide ou ReLU), pode aproximar qualquer função contínua em um subconjunto compacto de $\mathbb{R}^n$ com qualquer grau de precisão desejado. [George Cybenko \(1989\)](#).*

"Approximation by superpositions of a sigmoidal function". [Em: Mathematics of control, signals and systems 2.4](#), pp. 303–314

O Teorema da Aproximação Universal (UAT) II

Construindo Funções com “Blocos”

A prova se baseia na ideia de que podemos construir “blocos” de funções simples para aproximar qualquer forma complexa.

- 1 Uma função sigmóide $\sigma(z)$ é um “degrau suave”.
- 2 Com dois neurônios sigmóides (um com pesos positivos, outro com pesos negativos), podemos criar uma função “bump” (um pulso retangular suave).
- 3 Com muitos desses “bumps” de diferentes alturas, larguras e posições, podemos aproximar qualquer função contínua, como se estivessemos construindo uma paisagem com blocos de Lego.

UAT com ReLUs I

Passo 1: Aproximar Funções Contínuas com Funções Lineares por Partes

Qualquer função contínua $f(x)$ pode ser arbitrariamente bem aproximada por uma função contínua linear por partes (piecewise affine), $\hat{f}(x)$.

UAT com ReLUs II

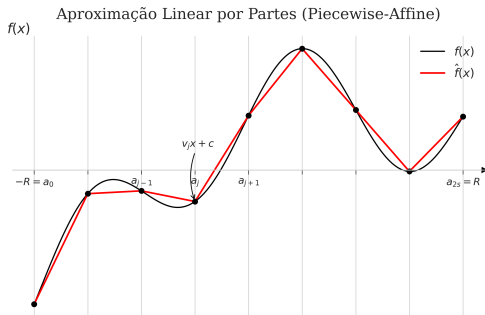


Figura: Aproximação de uma função suave (preto) por uma função linear por partes (vermelho).

UAT com ReLUs III

Passo 2: Representar Funções Lineares por Partes com ReLUs

A parte mais engenhosa é que **qualquer função contínua linear por partes pode ser representada exatamente** por uma rede com uma única camada oculta de neurônios ReLU.

A intuição é que cada neurônio ReLU, $(w_j x + b_j)_+$, cria uma “dobra” ou “quina” no espaço. Ao somar várias dessas “dobras” com pesos diferentes, podemos construir qualquer forma linear por partes.

UAT com ReLUs IV

Conclusão Lógica

- 1 Se redes com uma camada oculta de ReLUs podem representar **exatamente** qualquer função contínua linear por partes,
- 2 E funções contínuas lineares por partes podem **aproximar** qualquer função contínua,
- 3 Então, redes com uma camada oculta de ReLUs são **aproximadores universais**.

Se uma Camada é Universal, Por Que Usar Múltiplas? I

A Eficiência da Profundidade

O UAT garante que uma rede rasa *pode* representar qualquer função, mas não diz que ela o fará de forma **eficiente**.

- Para algumas funções, uma rede rasa pode precisar de um número **exponencial** de neurônios.
- Uma rede profunda pode representar a mesma função com um número muito menor de neurônios, distribuídos em camadas.

Se uma Camada é Universal, Por Que Usar Múltiplas? II

Teorema: Expressividade Exponencial da Profundidade

O número de regiões lineares distintas que uma rede profunda com ativações ReLU pode separar cresce exponencialmente com a profundidade L :

$$O\left(\left(\frac{K}{D}\right)^{D(L-1)} K^D\right)$$

Onde L é a profundidade, K o número de neurônios por camada e D a dimensão da entrada. Para K fixo, redes mais profundas são exponencialmente mais expressivas. [Guido F Montufar et al. \(2014\)](#). “On the number of linear regions of deep neural networks”. Em: *Advances in neural information processing systems 27*

Se uma Camada é Universal, Por Que Usar Múltiplas? III

Aprendizado Hierárquico de Features

A profundidade permite o aprendizado de uma **hierarquia de features**.

- **Camada 1:** Aprende features simples (e.g., bordas, cantos).
- **Camada 2:** Combina as features da camada 1 para aprender features mais complexas (e.g., olhos, narizes).
- **Camada 3:** Combina features da camada 2 para aprender objetos inteiros (e.g., faces).

Essa reutilização de features em múltiplos níveis de abstração é o que torna as redes profundas tão poderosas e eficientes em termos de parâmetros.

Se uma Camada é Universal, Por Que Usar Múltiplas? IV

Visualizar com Playground

Abrir Playground (XOR)

Funções de Ativação Comuns: Sigmóide I

Função Sigmóide

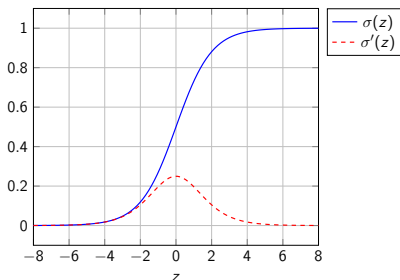
$$a = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Problema: Saturação dos gradientes (gradientes muito próximos de zero para entradas grandes ou pequenas).

Derivada:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) = a(1 - a)$$

Sigmóide e sua Derivada



Funções de Ativação Comuns: Tanh I

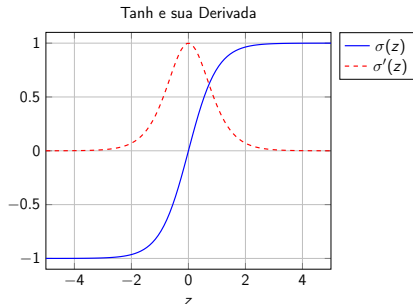
Tangente Hiperbólica (Tanh)

$$a = \sigma(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

É zero-centrada, o que pode acelerar a convergência. Mesmo problema da sigmóide.

Derivada:

$$\sigma'(z) = 1 - \sigma^2(z) = (1 + a)(1 - a)$$



Funções de Ativação Comuns: ReLU I

ReLU (Rectified Linear Unit)

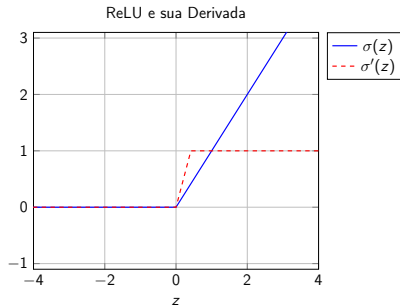
$$a = \sigma(z) = \max(0, z)$$

Eficiente e não satura para valores positivos. Pode “morrer” para entradas negativas.

Derivada

$$\sigma'(z) = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \end{cases}$$

(O gradiente em $z = 0$ é indefinido).



A Função Softmax para Classificação I

Generalizando a Sigmóide para Múltiplas Classes

Para $K > 2$ classes, queremos uma saída que represente uma distribuição de probabilidade. A função Softmax transforma um vetor de scores $z \in \mathbb{R}^K$ em um vetor de probabilidades $y \in [0, 1]^K$:

$$\text{softmax}(z)_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}$$

A Função Softmax para Classificação II

O Truque de Gumbel-Max

Como amostrar de uma distribuição categórica definida pelo softmax? O truque de Gumbel-Max nos permite fazer isso de forma diferenciável:

- 1 Adicione ruído de Gumbel a cada logit: $g_k = -\log(-\log(u_k))$, onde $u_k \sim \text{Uniform}(0, 1)$.
- 2 A amostra da distribuição categórica é simplesmente o índice do maior valor:

$$\text{amostra} = \arg \max_k (z_k + g_k)$$

Variações “suaves” (como a Gumbel-Softmax) são cruciais para treinar modelos generativos.

A Abstração do Grafo Computacional I

O que é um Grafo Computacional?

Qualquer expressão matemática, incluindo o *forward pass* de uma rede neural, pode ser decomposta em um Grafo Acíclico Dirigido (DAG), onde os nós representam variáveis e as arestas representam operações.

Backpropagation

Backpropagation é o nome que damos para este processo de duas etapas: um passe para frente para calcular os valores e um passe para trás para calcular os gradientes de forma eficiente.

A Abstração do Grafo Computacional II

Exemplo: Calculando o grafo para $J = (w \cdot x + b)^2$

Vamos decompor a computação e os gradientes passo a passo, usando os valores de exemplo $w = 3, x = 2, b = -1$.

1. Forward Pass (Cálculo dos Valores):

- *Passo 1:* Calculamos a multiplicação $a = w \cdot x \implies a = 6$.
- *Passo 2:* Calculamos a soma $z = a + b \implies z = 5$.
- *Passo 3:* Calculamos o resultado final $J = z^2 \implies J = 25$.

A Abstração do Grafo Computacional III

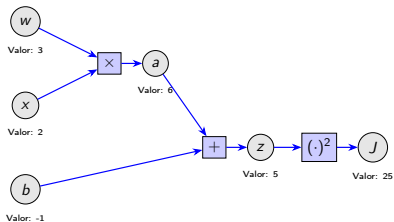
Exemplo: Calculando o grafo para $J = (w \cdot x + b)^2$

2. Backward Pass (Cálculo dos Gradientes): Começamos da saída e aplicamos a regra da cadeia para trás.

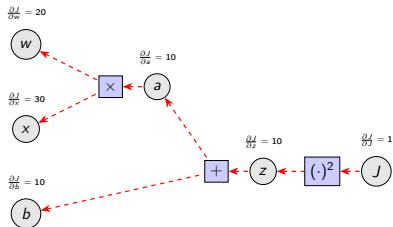
- *Gradiente inicial:* $\frac{\partial J}{\partial J} = 1$.
- *Gradiente em z :* $\frac{\partial J}{\partial z} = \frac{\partial J}{\partial J} \frac{\partial J}{\partial z} = 1 \cdot (2z) = 2 \cdot 5 = 10$.
- *Gradiente em a e b :* $\frac{\partial J}{\partial a} = \frac{\partial J}{\partial z} \frac{\partial z}{\partial a} = 10 \cdot 1 = 10$. E $\frac{\partial J}{\partial b} = \frac{\partial J}{\partial z} \frac{\partial z}{\partial b} = 10 \cdot 1 = 10$.
- *Gradiente em w e x :* $\frac{\partial J}{\partial w} = \frac{\partial J}{\partial a} \frac{\partial a}{\partial w} = 10 \cdot x = 10 \cdot 2 = 20$. E $\frac{\partial J}{\partial x} = \frac{\partial J}{\partial a} \frac{\partial a}{\partial x} = 10 \cdot w = 10 \cdot 3 = 30$.

Grafo Computacional: Exemplo Numérico I

1. Forward Pass: Calculando Valores

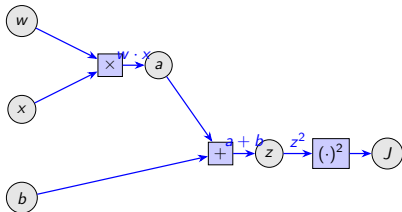


2. Backward Pass: Propagando Gradientes

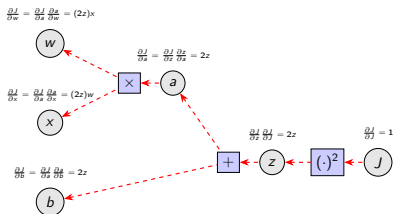


Grafo Computacional: Exemplo Simbólico I

1. Forward Pass: Calculando Variáveis



2. Backward Pass: Propagando Gradientes



O Algoritmo de Backpropagation: A Grande Ideia I

O Problema da Atribuição de Crédito

Como calcular o gradiente do custo J em relação aos pesos e vieses das camadas ocultas $(W^{(l)}, b^{(l)})$ para $l < L$?

O Algoritmo de Backpropagation: A Grande Ideia II

A Solução: Propagação de Erro para Trás

Backpropagation é uma aplicação eficiente da regra da cadeia que nos permite calcular esses gradientes de forma recursiva, começando da camada final e indo até a primeira.

A variável central é o "sinal de erro" para a camada l , definido como o gradiente do custo em relação à **entrada linear** $z^{(l)}$:

$$\delta^{(l)} \equiv \frac{\partial J}{\partial z^{(l)}}$$

Backpropagation Passo 1: Erro na Camada de Saída I

Calculando $\delta^{(L)}$

O primeiro passo é calcular o sinal de erro para a camada de saída L . Pela regra da cadeia:

$$\delta^{(L)} = \frac{\partial J}{\partial \mathbf{a}^{(L)}} \odot \frac{\partial \mathbf{a}^{(L)}}{\partial \mathbf{z}^{(L)}} = \frac{\partial J}{\partial \hat{\mathbf{y}}} \odot \sigma'_L(\mathbf{z}^{(L)})$$

Onde $\odot$ é o produto elemento a elemento (Hadamard).

Backpropagation Passo 1: Erro na Camada de Saída II

Caso Especial: Softmax + Cross-Entropy

Como vimos anteriormente, quando a camada de saída é Softmax e o custo é Cross-Entropy, esta expressão se simplifica elegantemente para:

$$\delta^{(L)} = \hat{y} - y$$

O sinal de erro inicial é simplesmente a diferença entre a previsão e a realidade.

Backpropagation Passo 2: Propagando o Erro I

Calculando $\delta^{(l)}$ a partir de $\delta^{(l+1)}$

A chave do backpropagation é a relação recursiva que nos permite calcular o erro de uma camada a partir do erro da camada seguinte:

$$\begin{aligned}\delta^{(l)} &= \frac{\partial J}{\partial \mathbf{z}^{(l)}} = \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{z}^{(l)}} \frac{\partial J}{\partial \mathbf{z}^{(l+1)}} \\ &= \left(\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \right) \delta^{(l+1)} \\ &= \left((\mathbf{W}^{(l+1)})^T \delta^{(l+1)} \right) \odot \sigma'_l(\mathbf{z}^{(l)})\end{aligned}$$

Backpropagation Passo 2: Propagando o Erro II

A Regra de Propagação

O sinal de erro em uma camada l é o sinal de erro da camada seguinte, propagado para trás através dos pesos $W^{(l+1)}$, e então multiplicado (elemento a elemento) pela derivada da função de ativação da camada l .

Backpropagation Passo 3: Gradientes dos Parâmetros I

Calculando $\nabla_{W^{(l)}} J$ e $\nabla_{b^{(l)}} J$

Uma vez que temos o sinal de erro $\delta^{(l)}$ para uma camada, calcular o gradiente de seus parâmetros é direto:

O gradiente para o viés $b^{(l)}$ é simplesmente o sinal de erro:

$$\frac{\partial J}{\partial b^{(l)}} = \delta^{(l)}$$

O gradiente para os pesos $W^{(l)}$ é o produto externo entre o sinal de erro e a ativação da camada anterior:

$$\frac{\partial J}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T$$

O Algoritmo de Backpropagation Resumido I

Para cada dado de treino (x, y) :

Forward:

- Defina $a^{(0)} = x$.
- Para $l = 1, \dots, L$, calcule:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = \sigma_l(z^{(l)})$$

O Algoritmo de Backpropagation Resumido II

Para cada dado de treino (x, y) :

Backward:

- Calcule o erro da saída: $\delta^{(L)} = \frac{\partial J}{\partial a^{(L)}} \odot \sigma'_L(z^{(L)})$.
- Para $l = L - 1, \dots, 1$, calcule (propague para trás):

$$\delta^{(l)} = \left((W^{(l+1)})^T \delta^{(l+1)} \right) \odot \sigma'_l(z^{(l)})$$

Cálculo dos Gradientes:

- Para cada camada $l = 1, \dots, L$:

$$\frac{\partial J}{\partial b^{(l)}} = \delta^{(l)} \quad \frac{\partial J}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T$$

Backprop para Softmax + Cross-Entropy I

O Cenário

Vamos derivar o gradiente para a camada de saída de um classificador multiclasse.

- **Entrada Linear (Logits):** $z = [z_1, \dots, z_K]$
- **Ativação (Probabilidades):** $\hat{y}_k = \text{softmax}(z)_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}$
- **Custo (Cross-Entropy):** $J = - \sum_{k=1}^K y_k \log(\hat{y}_k)$, onde y é o vetor one-hot.

Backprop para Softmax + Cross-Entropy II

Nosso Objetivo

Queremos encontrar o gradiente do custo em relação à entrada linear da camada de saída, $\frac{\partial J}{\partial z_i}$, usando a regra da cadeia:

$$\frac{\partial J}{\partial z_i} = \sum_{k=1}^K \frac{\partial J}{\partial \hat{y}_k} \frac{\partial \hat{y}_k}{\partial z_i}$$

Passo 1: A Derivada da Função Softmax I

A derivada da saída do softmax $\hat{y}_k$ em relação à sua entrada z_i tem dois casos:

Caso 1: $i = k$

$$\frac{\partial \hat{y}_k}{\partial z_k} = \frac{\exp(z_k) \sum_j \exp(z_j) - \exp(z_k) \exp(z_k)}{(\sum_j \exp(z_j))^2} = \hat{y}_k(1 - \hat{y}_k)$$

Caso 2: $i \neq k$

$$\frac{\partial \hat{y}_k}{\partial z_i} = \frac{0 - \exp(z_k) \exp(z_i)}{(\sum_j \exp(z_j))^2} = -\hat{y}_k \hat{y}_i$$

Passo 1: A Derivada da Função Softmax II

A Jacobiana da Softmax

Podemos combinar os dois casos de forma compacta usando o delta de Kronecker δ_{ik} :

$$\frac{\partial \hat{y}_k}{\partial z_i} = \hat{y}_k (\delta_{ik} - \hat{y}_i)$$

Passo 2: Juntando Tudo com a Regra da Cadeia I

Primeiro, a derivada do custo Cross-Entropy em relação à saída $\hat{y}_k$ é:

$$\frac{\partial J}{\partial \hat{y}_k} = -\frac{y_k}{\hat{y}_k}$$

Passo 2: Juntando Tudo com a Regra da Cadeia II

Agora, aplicamos a regra da cadeia:

$$\begin{aligned}\frac{\partial J}{\partial z_i} &= \sum_{k=1}^K \frac{\partial J}{\partial \hat{y}_k} \frac{\partial \hat{y}_k}{\partial z_i} = \sum_{k=1}^K \left(-\frac{y_k}{\hat{y}_k} \right) (\hat{y}_k (\delta_{ik} - \hat{y}_i)) \\ &= - \sum_{k=1}^K y_k (\delta_{ik} - \hat{y}_i) \\ &= - \left(\sum_{k=1}^K y_k \delta_{ik} - \sum_{k=1}^K y_k \hat{y}_i \right) \\ &= - \left(y_i - \hat{y}_i \sum_{k=1}^K y_k \right) \quad (\text{Lembre-se que } \sum y_k = 1) \\ &= -(y_i - \hat{y}_i) = \hat{y}_i - y_i\end{aligned}$$

Passo 2: Juntando Tudo com a Regra da Cadeia III

O Resultado Elegante

O gradiente do custo Cross-Entropy com Softmax em relação à entrada linear z_i é simplesmente a diferença entre a predição e o valor verdadeiro:

$$\frac{\partial J}{\partial z_i} = \hat{y}_i - y_i$$

Passo 3: Propagando o Gradiente para Camadas Anteriores I

Propagando o Sinal de Erro

Uma vez que temos o "sinal de erro" na camada de saída, $\frac{\partial J}{\partial z^{(L)}} = \hat{y} - y$, podemos propagá-lo para a camada oculta anterior. O gradiente em relação à ativação da camada oculta h_j é:

$$\frac{\partial J}{\partial h_j} = \sum_k \frac{\partial J}{\partial z_k^{(L)}} \frac{\partial z_k^{(L)}}{\partial h_j} = \sum_k (\hat{y}_k - y_k) w_{kj}^{(L)}$$

Onde $w_{kj}^{(L)}$ é o peso que conecta o neurônio oculto j ao neurônio de saída k .

Passo 3: Propagando o Gradiente para Camadas Anteriores II

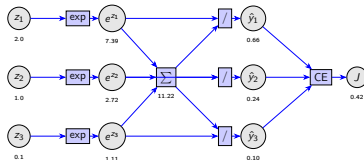
E o Gradiente dos Pesos?

O gradiente para um peso $w_{ji}^{(H)}$ na camada oculta é então calculado como:

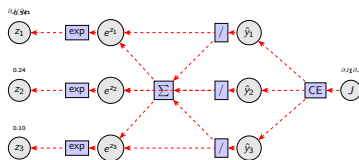
$$\frac{\partial J}{\partial w_{ji}^{(H)}} = \frac{\partial J}{\partial h_j} \frac{\partial h_j}{\partial z_j^{(H)}} \frac{\partial z_j^{(H)}}{\partial w_{ji}^{(H)}} = \left(\sum_k (\hat{y}_k - y_k) w_{kj}^{(L)} \right) \sigma'(z_j^{(H)}) x_i$$

Grafo Computacional: Exemplo Softmax I

1. Forward Pass



2. Backward Pass



Backpropagation vs. Diferenciação Automática (AD) I

Uma Distinção Importante

- **Backpropagation:** É o **algoritmo** que especifica como aplicar a regra da cadeia para calcular gradientes em uma rede neural em camadas.
- **Diferenciação Automática (AD):** É a **técnica** computacional geral para calcular derivadas de qualquer programa de computador.

Backpropagation vs. Diferenciação Automática (AD) II

Modo Reverso vs. Modo Direto

AD tem dois modos principais:

- **Modo Reverso (Backprop):** Propaga derivadas da saída para a entrada. Custo de um *forward pass* + um *backward pass*. Ideal para funções com muitas entradas e poucas saídas (como a função de custo de uma rede).
- **Modo Direto:** Propaga derivadas da entrada para a saída. Requer um passe para cada variável de entrada. Ineficiente para redes neurais, mas útil em outros contextos.

Mergulho Raso: Modo Direto e Números Duais I

Como o Modo Direto Funciona?

O modo direto calcula o valor da função e sua derivada *simultaneamente* em uma única passagem para frente. A base matemática para isso são os **Números Duais**.

Mergulho Raso: Modo Direto e Números Duais II

A Magia dos Números Duais

Um número dual tem a forma $a + b\epsilon$, onde ϵ é um “infinitesimal” com a propriedade $\epsilon^2 = 0$.

Se aplicarmos uma função f a um número dual $x = x_0 + \epsilon$, a expansão em série de Taylor nos dá:

$$f(x_0 + \epsilon) = f(x_0) + f'(x_0)\epsilon + \frac{f''(x_0)}{2!}\epsilon^2 + \dots$$

Como $\epsilon^2 = 0$ (e potências maiores também), a série se trunca:

$$f(x_0 + \epsilon) = \underbrace{f(x_0)}_{\text{Parte Real}} + \underbrace{f'(x_0)}_{\text{Parte Dual}} \epsilon$$

Mergulho Raso: Modo Direto e Números Duais III

Exemplo

Para $f(x) = x^2$ em $x_0 = 3$, avaliamos em $3 + 1\epsilon$:

$$f(3 + \epsilon) = (3 + \epsilon)^2 = 3^2 + 2 \cdot 3 \cdot \epsilon + \epsilon^2 = 9 + 6\epsilon$$

A parte real nos dá o valor da função ($f(3) = 9$) e a parte dual nos dá a derivada ($f'(3) = 6$).

Implementações Modernas de AD I

PyTorch: Grafo Dinâmico

A abordagem do PyTorch é **imperativa** e baseada em “fitas” (tapes).

- Quando operações são executadas em tensores com `'requires_grad=True'`, PyTorch grava essas operações em um grafo dinâmico.
- Chamar `'.backward()'` em um escalar (como o custo) percorre este grafo ao contrário, aplicando a regra da cadeia para acumular os gradientes nos tensores folha.

Implementações Modernas de AD II

JAX: Transformação Funcional

A abordagem do JAX é **funcional** e baseada em transformações.

- Você escreve funções Python puras que operam em arrays NumPy.
- A função `'jax.grad()'` é uma transformação: ela recebe sua função f e retorna uma *nova função* `'grad_f'` que calcula o gradiente de f .
- JAX pode compor transformações (`'jit'`, `'vmap'`, `'grad'`), permitindo otimizações poderosas.

Implementações Modernas de AD III

O Ponto em Comum

Ambos os frameworks implementam **AD em modo reverso** de forma altamente otimizada. Nosso trabalho é definir o *forward pass* (o modelo); eles cuidam do *backward pass* (os gradientes).

O Grafo Computacional como Otimização I

O Grafo como um Sistema de Equações

O *forward pass* de uma rede pode ser visto como um grande sistema de equações. Cada nó v_i no grafo computacional representa uma variável, e seu valor é definido por uma função de seus pais, $Pa(v_i)$. Isso nos dá um conjunto de restrições de igualdade que devem ser satisfeitas:

$$v_i - f_i(Pa(v_i)) = 0, \quad \forall i$$

O objetivo é calcular o valor do nó final, $v_N = J$, sujeito a todas essas restrições.

O Grafo Computacional como Otimização II

A Conexão com Otimização

Encontrar os gradientes ($\frac{\partial J}{\partial v_k}$) é equivalente a perguntar: “Se eu pudesse violar um pouco a restrição do nó v_k , qual seria o impacto no valor final J ?” Esta é exatamente a pergunta que os **Multiplicadores de Lagrange** foram projetados para responder.

Derivação via Multiplicadores de Lagrange I

Construindo o Lagrangeano

Para resolver este problema, construímos o Lagrangeano, onde cada restrição ganha um multiplicador de Lagrange λ_i :

$$\mathcal{L} = v_N - \sum_{i=1}^N \lambda_i (v_i - f_i(Pa(v_i)))$$

Derivação via Multiplicadores de Lagrange II

A Condição de Otimalidade

No ponto ótimo, a derivada do Lagrangeano em relação a cada variável v_k deve ser zero. Focando em um nó intermediário v_k ($k < N$):

$$\frac{\partial \mathcal{L}}{\partial v_k} = 0 - \lambda_k + \sum_{j \in Ch(v_k)} \lambda_j \frac{\partial f_j(Pa(v_j))}{\partial v_k} = 0$$

Onde $Ch(v_k)$ são os “filhos” de v_k no grafo. Reorganizando, obtemos:

$$\lambda_k = \sum_{j \in Ch(v_k)} \lambda_j \frac{\partial v_j}{\partial v_k}$$

Derivação via Multiplicadores de Lagrange III

A Identidade Fundamental

Esta equação é a **regra da cadeia**! Ela nos diz que o multiplicador λ_k é a soma dos multiplicadores dos nós filhos, ponderados pela derivada local. Isso revela a identidade fundamental: o multiplicador de Lagrange λ_k é precisamente o gradiente do nó final em relação ao nó v_k :

$$\lambda_k \equiv \frac{\partial v_N}{\partial v_k} = \frac{\partial J}{\partial v_k}$$

O Backpropagation é, portanto, um algoritmo para calcular eficientemente os multiplicadores de Lagrange deste problema de otimização.

Derivação via Multiplicadores de Lagrange IV

Leitura Adicional

Para aprofundar na distinção entre Backprop e AD:

Justin Domke (2011). *Automatic Differentiation and Neural Networks*. Lecture Notes, Statistical Machine Learning. URL: https://people.cs.umass.edu/~domke/courses/sml2011/08autodiff_nnets.pdf

Tim Vieira (ago. de 2017). *Backprop is not just the chain rule*. Blog Post. URL: <https://timvieira.github.io/blog/post/2017/08/18/backprop-is-not-just-the-chain-rule/> (acesso em 15/09/2025)

Visualizando o Grafo Lagrangeano

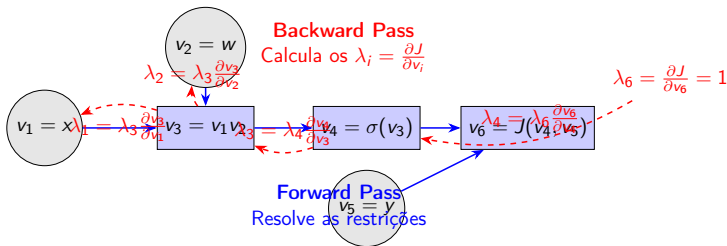


Figura: Cada nó tem uma equação de restrição. O backpropagation (modo reverso) calcula os multiplicadores de Lagrange (os gradientes $\partial J / \partial v_i$) de trás para frente.

Encerramento e Perguntas I

Resumo da Aula

- Aprofundamos no poder dos MLPs com o Teorema da Aproximação Universal e a noção de expressividade.
- Exploramos funções de ativação e custo, incluindo o Softmax e suas variantes esparsas.
- Formalizamos o aprendizado como a Minimização do Risco Empírico.
- Derivamos o Backpropagation e o conectamos à técnica geral da Diferenciação Automática.

Encerramento e Perguntas II

Para a Próxima Aula: Laboratório Prático!

- **Tópico:** Treinando um MLP do Zero.
- **Objetivo:** Juntar todas as peças — MLP, função de custo, Backpropagation e Descida de Gradiente — para resolver um problema de classificação não-linear em um notebook.

Perguntas?

Referências I

-  Bach, Francis (2025). *Learning Theory from First Principles*.
URL: https://www.di.ens.fr/~fbach/ltfp_book.pdf.
-  Bishop, Christopher M. (2006). *Pattern Recognition and Machine Learning*. Springer.
-  Bishop, Christopher M. e Hugh Bishop (2023). *Deep Learning: Foundations and Concepts*. Springer.
-  Buchanan, Sam et al. (ago. de 2025). *Learning Deep Representations of Data Distributions*.
<https://ma-lab-berkeley.github.io/deep-representation-learning-book/>. Online.
-  Cybenko, George (1989). “Approximation by superpositions of a sigmoidal function”. Em: *Mathematics of control, signals and systems* 2.4, pp. 303–314.

Referências II

-  Domke, Justin (2011). *Automatic Differentiation and Neural Networks*. Lecture Notes, Statistical Machine Learning. URL: https://people.cs.umass.edu/~domke/courses/sml2011/08autodiff_nnets.pdf.
-  Gefer, Amanda (fev. de 2015). “The Man Who Tried to Redeem the World with Logic”. Em: *Nautilus*. URL: <https://nautil.us/the-man-who-tried-to-redeem-the-world-with-logic-235253/>.
-  Grünwald, Peter D. (2007). *The Minimum Description Length Principle*. The MIT Press.
-  Jordan, Michael I (2019). “Artificial intelligence—the revolution hasn’t happened yet”. Em: *Harvard Data Science Review* 1.1.
-  Krause, Andreas e Jonas Hübner (2025). *Probabilistic Artificial Intelligence*. arXiv: 2502.05244 [cs.AI].

Referências III

-  Liu, Yuxi (jan. de 2024). *The Perceptron Controversy*. Website. URL: <https://yuxi.ml/essays/posts/perceptron-controversy/> (acesso em 10/09/2025).
-  Martins, André FT e Ramón F Astudillo (2016). “From softmax to sparsemax: A sparse model of attention and multi-label classification”. Em: *International conference on machine learning*. PMLR, pp. 1614–1623.
-  McCulloch, Warren S. e Walter Pitts (1943). “A Logical Calculus of the Ideas Immanent in Nervous Activity”. Em: *The Bulletin of Mathematical Biophysics* 5.4, pp. 115–133. DOI: [10.1007/bf02478259](https://doi.org/10.1007/bf02478259).

Referências IV

-  Mercer, James (1909). “Functions of positive and negative type, and their connection with the theory of integral equations”. *Em: Philosophical transactions of the Royal Society of London. Series A* 209.441-458, pp. 415–446.
-  Minsky, Marvin e Seymour A Papert (1969). *Perceptrons: An introduction to computational geometry*. MIT press.
-  Montufar, Guido F et al. (2014). “On the number of linear regions of deep neural networks”. *Em: Advances in neural information processing systems* 27.
-  Murphy, Kevin Patrick (2023). *Probabilistic Machine Learning: Advanced Topics*. The MIT Press.
-  Oja, Erkki (1982). “A simplified neuron model as a principal component analyzer”. *Em: Journal of mathematical biology* 15.3, pp. 267–273.

Referências V

-  Prince, Simon J.D. (2023). *Understanding Deep Learning*. The MIT Press.
-  Rasmussen, Carl Edward e Christopher K. I. Williams (2006). *Gaussian processes for machine learning*. The MIT Press.
-  Rosenblatt, Frank (1958). "The perceptron: a probabilistic model for information storage and organization in the brain.". Em: *Psychological review* 65.6, p. 386.
-  Schölkopf, Bernhard e Alexander J Smola (2002). *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press.
-  Shalev-Shwartz, Shai e Shai Ben-David (2014). *Understanding machine learning: From theory to algorithms*. Cambridge university press.

Referências VI

-  Shannon, C. E. (1988). “Programming a computer for playing chess”. Em: *Computer Chess Compendium*. Berlin, Heidelberg: Springer-Verlag, pp. 2–13. ISBN: 0387913319.
-  Shannon, Claude E. (1948). *A Mathematical Theory of Communication*. Vol. 27, pp. 379–423, 623–656.
-  Turing, Alan M. (1969). “Intelligent Machinery”. Em: *Machine Intelligence 5*. Ed. por Bernard Meltzer e Donald Michie. Edinburgh University Press, pp. 3–23.
-  Vapnik, Vladimir (1998). *Statistical learning theory*. Wiley.
-  Vieira, Tim (ago. de 2017). *Backprop is not just the chain rule*. Blog Post. URL: <https://timvieira.github.io/blog/post/2017/08/18/backprop-is-not-just-the-chain-rule/> (acesso em 15/09/2025).